



UNIVERSIDADE DE BRASÍLIA
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

CIC0124 REDES DE COMPUTADORES - TRABALHO 2
MINI-ONE DRIVE

Autores:

Felipe Lauterjung Caselli, 24/1032401
Laíssa Beatriz Soares da Silva, 22/2032982
Marcio Vinicius da Silva Guimaraes, 24/2001553
Pedro Fernandes de Oliveira, 23/1006177

Professor:

Roberto Rodrigues Filho, Ph.D.

Brasília/DF
2026

1. Introdução

O projeto foi desenvolvido com o objetivo de aplicar, na prática, os conceitos estudados na disciplina de Redes de Computadores por meio da implementação de um serviço distribuído de sincronização de arquivos. Para isso, foram definidos mecanismos de descoberta de nós, integração dinâmica de dispositivos, propagação de alterações e sincronização automática entre os participantes da rede, conforme os requisitos estabelecidos na proposta do trabalho.

2. Visão geral da solução

O *Mini-OneDrive* inspira-se em plataformas modernas de armazenamento em nuvem, substituindo a tradicional arquitetura Cliente-Servidor por uma rede 100% descentralizada. A topologia permite que qualquer nó integre ou abandone a rede de forma dinâmica, garantindo que a sincronização dos dados entre os pares ocorra de forma autônoma.

2.1. Principais funcionalidades

- **Descoberta Dinâmica (UDP Broadcast):** Detecção automática de nós na rede local (LAN), operando sem a necessidade de um servidor centralizado de registros ou da configuração manual de endereços IP.
- **Tolerância a Falhas (*Heartbeat*):** Mecanismo de monitoramento contínuo do estado da rede. Em caso de desconexão abrupta de um nó, a aplicação detecta a ausência por limite de tempo (*timeout*) e atualiza a topologia em tempo real.
- **Monitoramento do Sistema de Arquivos:** Registro e acompanhamento dinâmico das operações de criação, modificação e exclusão de arquivos nos diretórios locais designados.
- **Resolução de Conflitos:** Implementação de controle de causalidade para gerenciar edições simultâneas *offline*, prevenindo a perda de dados através da criação de versões de conflito.
- **Sincronização Delta (TCP):** Protocolo de transferência otimizada de dados, baseado na segmentação de arquivos (*chunks*) e na verificação de integridade através de algoritmos de *hash* (SHA-256).

2.2. Tecnologias utilizadas

A arquitetura do projeto priorizou a minimização de dependências externas, recorrendo exclusivamente aos recursos nativos da linguagem para a lógica principal:

- **Linguagem de Programação:** Python 3.11 (Uso das bibliotecas padrão: socket, threading, json, hashlib, os).
- **Ambiente e Virtualização:** Docker e Docker Compose.
- **Protocolos de Comunicação:** UDP (Gestão de topologia e descoberta) e TCP (Transferência confiável de dados).

3. Instruções para execução

A solução é executado em um ambiente isolado através do Docker, o que permite a simulação de uma rede composta por múltiplos dispositivos na mesma máquina. Dessa forma, é necessário que o Docker esteja instalado no sistema *host* e o Docker Compose esteja configurado.

3.1. Procedimento de inicialização

■ Clonar o repositório

```
git clone  
[https://github.com/pedrofernandss/simplified_onedrive.git](https://github.com/pedrofernandss/simplified_onedrive.git)  
cd simplified_onedrive
```

- #### ■ Iniciar a topologia base:
- Para instanciar os dois primeiros nós e estabelecer a rede primária, execute:

```
docker compose up node_a node_b --build
```

Nota: O terminal apresentará os *logs* da inicialização, demonstrando a descoberta mútua via protocolo UDP.

- #### ■ Adição dinâmica de nós:
- Para verificar a capacidade de integração em tempo real, abra uma nova janela de terminal no mesmo diretório e inicie um terceiro nó:

```
docker compose up node_c
```

Nota: O nó C emitirá um pacote de anúncio (*broadcast*), sendo imediatamente reconhecido e integrado pelos nós A e B em suas tabelas de pares.

- **Encerrar o sistema:** Para terminar a execução e remover os recursos temporários de rede, utilize o comando:

```
docker compose down
```

Nota: O comando acima encerra todos os contêineres e remove os recursos temporários da rede criados durante a execução.

4. Arquitetura do projeto

O projeto foi desenvolvido com uma arquitetura peer-to-peer simplificada, em que cada nó executa a mesma base de código e atua de forma independente dentro da rede local. A estrutura da aplicação é organizada em módulos com responsabilidades bem definidas, o que facilita a compreensão, manutenção e extensão do funcionamento.

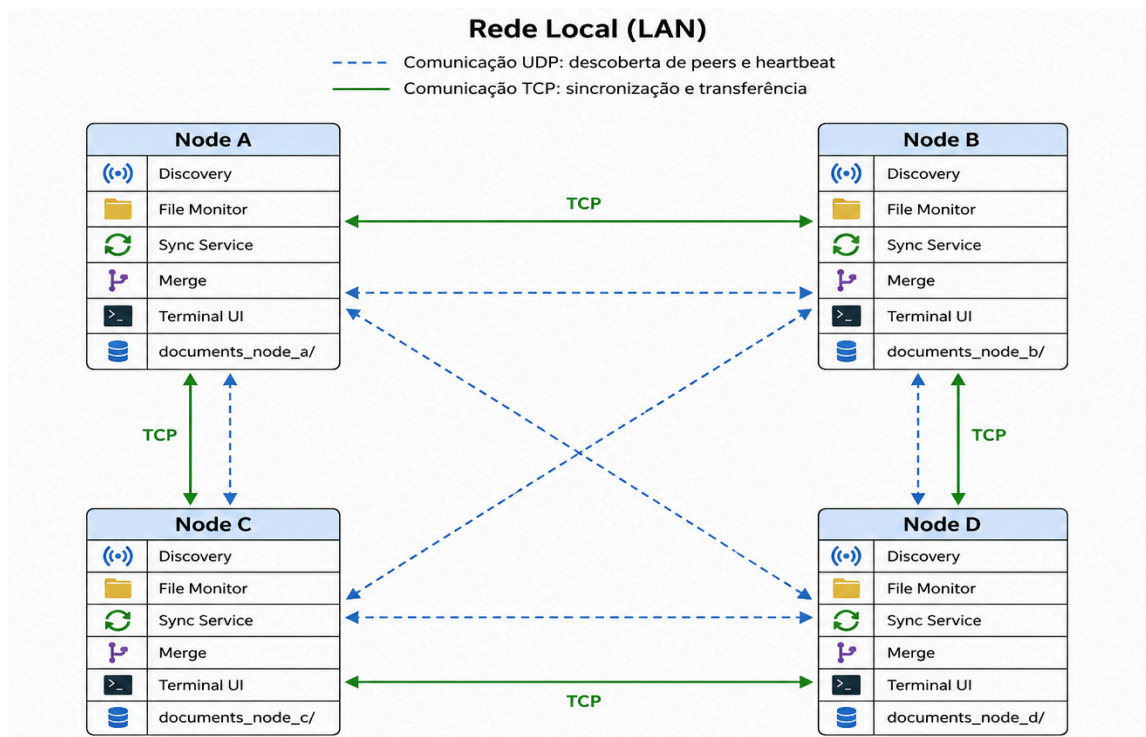


Figura 1 - Arquitetura do projeto.

A aplicação inicia no arquivo `main.py`, que é o ponto de entrada do programa. A partir dele, é criado um objeto `Node`, responsável por coordenar os principais serviços do nó. Esse componente atua como orquestrador do projeto, integrando a descoberta de outros peers, o monitoramento do diretório local e a sincronização de arquivos.

A descoberta de nós na rede é realizada pelo módulo `discovery.py`, que utiliza comunicação UDP por broadcast para anunciar a presença do nó e identificar outros participantes ativos. Esse serviço mantém uma lista de peers conhecidos e remove aqueles que ficam inativos após um período de timeout.

O monitoramento de arquivos é feito pelo módulo `file_monitor.py`. Ele observa periodicamente o diretório compartilhado, detectando a criação, alteração ou exclusão de arquivos. Sempre que uma mudança é identificada, o monitor gera um evento que é processado pela camada de sincronização.

A sincronização entre nós é implementada pelo módulo `sync_service.py`, que utiliza TCP para trocar mensagens e arquivos. Esse serviço oferece funcionalidades para listar os arquivos disponíveis em um peer, solicitar o conteúdo de um arquivo, enviar alterações e propagar exclusões. O fluxo de sincronização permite que um nó receba arquivos ausentes ou atualizados quando entra na rede ou quando outro nó altera um conteúdo já compartilhado.

A resolução de conflitos é feita por meio do módulo `merge.py`, que tenta combinar alterações textuais quando elas são compatíveis. Caso a alteração de dois nós gere divergências incompatíveis, o mecanismo marca o arquivo com blocos de conflito, permitindo que o usuário resolva a situação manualmente antes de continuar a sincronização.

Por fim, a interface de terminal, implementada em `terminal_ui.py`, padroniza os logs exibidos no console, permitindo acompanhar em tempo real as operações de descoberta, sincronização e comunicação entre os nós.

A arquitetura do sistema pode ser resumida em três pilares principais: descoberta de peers, monitoramento de arquivos e sincronização via TCP.

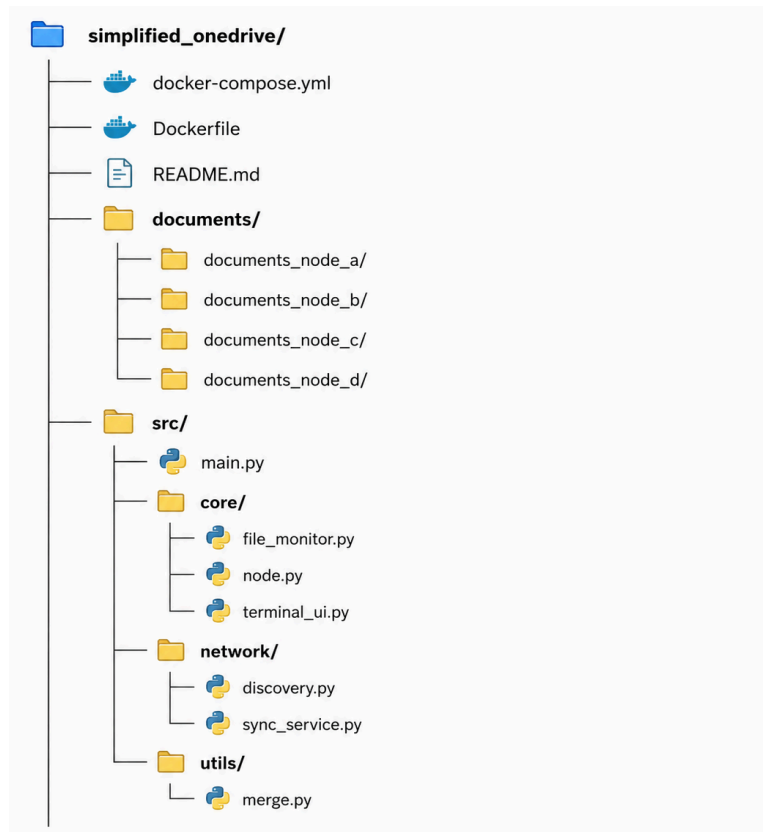


Figura 2 - Estrutura de pastas.

5. Protocolos de comunicação

A comunicação entre os nós do serviço é realizada por meio de dois protocolos principais: UDP, utilizado para descoberta de peers, e TCP, utilizado para transferência de arquivos e sincronização de dados. Essa combinação permite que o sistema seja simples, eficiente e compatível com a arquitetura peer-to-peer implementada.

O serviço de descoberta emprega UDP. Cada nó envia periodicamente mensagens de broadcast para a rede local, informando sua identidade e presença. Os demais nós recebem essas mensagens e atualizam sua lista de peers ativos. Essa abordagem permite que novos nós sejam identificados automaticamente sem a necessidade de configuração manual. Além disso, o mecanismo utiliza um mecanismo de timeout para considerar um nó desconectado caso ele não envie mensagens por um determinado período.

Já a sincronização ocorre sobre conexões TCP para a comunicação mais confiável entre os nós, principalmente nas operações de sincronização. Quando um nó precisa trocar arquivos ou verificar o conteúdo de um diretório remoto, ele estabelece uma conexão TCP com o peer correspondente. A partir dessa conexão, são enviados comandos como listagem de arquivos, solicitação de download, envio de conteúdo e remoção de arquivos. O uso do TCP é adequado nesse contexto porque garante a entrega ordenada e confiável dos dados, o que é essencial para manter os diretórios sincronizados.

No projeto, o fluxo de comunicação funciona de forma distribuída: um nó que detecta uma alteração local publica essa mudança para os peers conhecidos; em seguida, o peer remoto pode aceitar, aplicar ou resolver a alteração conforme a situação. Esse modelo permite que o sistema mantenha consistência entre os nós mesmo em cenários de atualização simultânea.

Além disso, a solução também faz uso de mensagens estruturadas em formato JSON, acompanhadas de um cabeçalho com tamanho definido. Essa estratégia organiza as comunicações e facilita a interpretação das ações enviadas entre os nós. O protocolo implementado suporta operações como listar arquivos, obter um arquivo específico, enviar uma nova versão, e apagar arquivos remotamente.

Em resumo, o UDP permite a descoberta automática dos nós, enquanto o TCP garante a transferência confiável de arquivos e comandos de sincronização entre os peers.

6. Mecanismos de sincronização

A sincronização do serviço é realizada por meio de um conjunto de mecanismos que permitem manter os diretórios dos nós consistentes, mesmo diante de alterações locais e remotas. O processo começa com o monitoramento contínuo do diretório compartilhado, que detecta a criação, modificação ou exclusão de arquivos. Quando uma mudança é identificada, o nó responsável notifica a camada de sincronização, que repassa a alteração para os demais peers conectados.

A propagação das modificações ocorre por meio de conexões TCP, nas quais um nó envia o conteúdo atualizado do arquivo para os demais participantes da rede. Para verificar se duas versões correspondem ao mesmo conteúdo, o sistema calcula hashes antes e após as alterações, permitindo verificar se a versão recebida corresponde à versão esperada. Além disso, a aplicação armazena o hash anterior da modificação, o que ajuda a identificar situações em que a alteração remota pode ser aplicada com segurança.

Em caso de divergência entre versões de um mesmo arquivo, o projeto emprega mecanismos de detecção de conflitos. Quando duas alterações incompatíveis são detectadas, o sistema tenta realizar um merge textual automático. Caso essa resolução não seja segura, o conteúdo é marcado com blocos de conflito, indicando que a correção deve ser feita manualmente pelo usuário. Esse procedimento evita a perda de informação e torna o processo de sincronização mais robusto.

Outro mecanismo importante é a propagação de exclusões. Quando um arquivo é removido em um nó, a informação é enviada aos outros peers, que também apagam a cópia local correspondente. Dessa forma, a rede mantém uma visão consistente dos arquivos compartilhados.

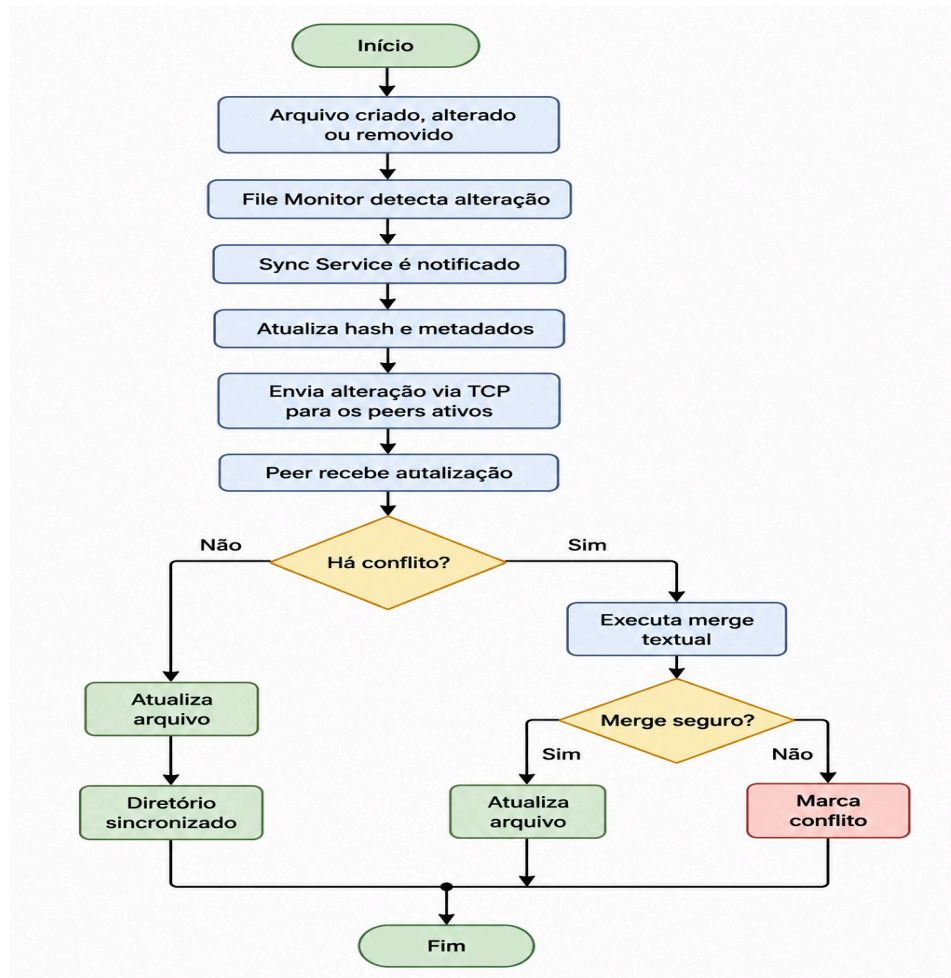


Figura 3 - Fluxograma dos mecanismos de sincronização.

7. Implementação

A implementação foi organizada em módulos com responsabilidades específicas, o que facilita a manutenção e a compreensão do projeto. A aplicação é iniciada por um ponto central, que instancia os componentes responsáveis por descoberta de peers, monitoramento de arquivos e sincronização. Cada nó executa essa estrutura de forma independente, mas interage com os demais nós por meio de protocolos de rede.

A descoberta de nós é implementada por meio de broadcast UDP, permitindo que os participantes da rede se encontrem automaticamente. O monitoramento do diretório local é realizado por uma thread periódica, que verifica alterações de arquivos e dispara eventos quando há mudanças. Já a sincronização é conduzida por um serviço TCP, responsável por estabelecer conexões com outros

nós e executar operações como listagem de arquivos, download, envio de conteúdo e remoção.

O projeto também conta com mecanismos de interface de terminal, que exibem logs padronizados relacionados à descoberta, sincronização, conflitos e operações sobre arquivos. Essa camada de visualização contribui para a observabilidade durante a execução. A implementação foi pensada para ser executada em ambiente Docker, o que simplifica a criação de múltiplos nós e a simulação de uma rede distribuída em um ambiente local.

8. Testes e resultados

Para validar o funcionamento, foram realizados testes manuais em ambiente local com múltiplos nós em execução por meio do Docker Compose. O objetivo foi verificar se a descoberta de peers, a sincronização de arquivos e o tratamento de conflitos ocorriam de forma adequada entre os diferentes participantes da rede.

No primeiro teste, foi iniciada a comunicação entre os nós e observada a descoberta automática dos peers. Os nós conseguiram se identificar por meio de mensagens broadcast UDP, e a lista de peers ativos foi atualizada corretamente ao longo da execução. Esse teste confirmou a capacidade do projeto de localizar outros nós sem configuração manual. O comando utilizado para subir os nós foi:

```
docker compose up node_a node_b --build
```

Em seguida, foram realizados testes de criação e modificação de arquivos. Para verificar a propagação de uma nova criação, foi executado o seguinte comando no nó A:

```
docker compose exec node_a sh -c "echo 'arquivo criado no node_a'
> /app/documents/teste_a.txt"
```

Depois, a replicação do arquivo foi confirmada nos demais nós com:

```
docker compose exec node_b cat /app/documents/teste_a.txt
```

```
docker compose exec node_c cat /app/documents/teste_a.txt
```

Para testar a modificação de um arquivo já existente, foi alterado o conteúdo no nó B com o comando:

```
docker compose exec node_b sh -c "echo 'alterado pelo node_b' >> /app/documents/teste_a.txt"
```

Em seguida, a atualização foi verificada no nó A:

```
docker compose exec node_a cat /app/documents/teste_a.txt
```

Também foi testada a remoção de arquivos. Para isso, o arquivo foi excluído no nó A utilizando:

```
docker compose exec node_a rm /app/documents/teste_a.txt
```

E a remoção foi confirmada no nó B com:

```
docker compose exec node_b ls /app/documents
```

Por fim, foram avaliados os cenários de conflito. Quando dois nós alteravam o mesmo arquivo de forma incompatível, o sistema identificava a divergência e tentava realizar um merge textual. Em casos em que a resolução automática não era segura, o arquivo era marcado com blocos de conflito para que a correção fosse feita manualmente. Esse comportamento confirmou a importância do tratamento de conflitos no projeto e a utilidade do mecanismo implementado.

9. Dificuldades encontradas

Durante o desenvolvimento do projeto, foram encontradas algumas dificuldades técnicas e conceituais. A principal delas foi a implementação da sincronização entre nós de forma confiável, considerando a possibilidade de alterações simultâneas em um mesmo arquivo. Esse problema exigiu a adoção de

mecanismos de hash, detecção de conflitos e merge textual, a fim de evitar sobrescritas indevidas e perda de informação.

Outra dificuldade foi a integração entre os diferentes módulos da aplicação. Como a aplicação é composta por componentes responsáveis por descoberta, monitoramento e sincronização, foi necessário garantir que a comunicação entre eles fosse bem coordenada. Isso incluiu o correto tratamento de eventos gerados pelo monitoramento de arquivos e a propagação dessas mudanças para os serviços de rede.

Além disso, a implementação da comunicação em rede com UDP e TCP demandou atenção especial ao tratamento de erros, timeouts e conexões. Em ambientes de rede local, pequenas variações de latência e timing podem afetar a execução, o que exigiu cuidados adicionais na implementação para evitar comportamentos instáveis.

Também houve dificuldade relacionada ao desenvolvimento de uma estratégia de conflitos que fosse útil, mas sem tornar o mecanismo excessivamente complexo. A solução adotada, baseada em merge textual e marcação de conflitos, mostrou-se adequada para o escopo do projeto, embora ainda tenha limitações em cenários mais complexos.

10. Conclusão

O desenvolvimento deste projeto permitiu aplicar, na prática, diversos conceitos estudados na disciplina de Redes de Computadores por meio da implementação de um sistema distribuído para sincronização de arquivos em uma arquitetura peer-to-peer. Durante a construção da solução foram implementados mecanismos de descoberta de nós na rede, comunicação utilizando os protocolos UDP e TCP, sincronização automática de arquivos e tratamento básico de conflitos, atendendo aos principais requisitos propostos para o trabalho.

Os testes realizados demonstraram que os nós conseguem ingressar dinamicamente na rede, identificar os demais participantes e manter o conteúdo das pastas compartilhadas sincronizado de forma automática. A integração entre os mecanismos de descoberta, monitoramento de alterações e propagação dos

arquivos permitiu que o projeto funcionasse de maneira consistente em diferentes cenários de uso em rede local.

Embora ainda existam limitações, principalmente em situações envolvendo alterações simultâneas em um mesmo arquivo ou em ambientes com um número maior de participantes, os resultados obtidos demonstram que a arquitetura adotada atende ao objetivo da proposta e constitui uma base para futuras evoluções da aplicação. Entre as melhorias possíveis estão o aprimoramento da resolução de conflitos, a otimização da sincronização e a inclusão de mecanismos adicionais de segurança e autenticação.

De forma geral, o trabalho atingiu os objetivos estabelecidos, permitindo compreender os desafios envolvidos na implementação de sistemas distribuídos e protocolos de comunicação, além de consolidar, na prática, conhecimentos relacionados à comunicação em rede, sincronização de dados e funcionamento de aplicações P2P.

11. Referências

KUROSE, James F.; ROSS, Keith W. *Computer Networking: A Top-Down Approach*. 9. ed. Hoboken: Pearson, 2021.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. *Sistemas distribuídos: conceitos e projeto*. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER. Docker overview. Disponível em: <https://docker.com>. Acesso em: 28 jun. 2026.

PYTHON SOFTWARE FOUNDATION. *socket — Low-level networking interface*. Python 3 Documentation. Disponível em: <https://docs.python.org/3/library/socket.html>. Acesso em: 28 jun. 2026.

UNIVERSIDADE DE BRASÍLIA. Departamento de Ciência da Computação. [Trabalho 2 - T2](#). Material disponibilizado na disciplina de Redes de Computadores. Acesso em: 28 de junho 2026.